GUÍA PRÁCTICA LAB 03 Y PROMPTS OPTIMIZADOS • EDICIÓN RETAIL OMNICANAL (RETAIL PULSE)

Delivered by Tryolabs • Powered by Google Antigravity & Gemini Enterprise

TRYOLABS • GOOGLE ANTIGRAVITY WORKSHOP • LAB 03 (ES)

Laboratorio 03: Servidor MCP de Retail, Hooks Deterministas de Margen y Gobernanza de Seguridad Empresarial

Duración: 45 Minutos | **Perfil**: Track Técnico y Comercial | **Prerrequisitos**: Laboratorio 02 Completado

1. Escenario de Retail y Contexto de Negocio

Incluso después de codificar reglas y skills (Laboratorio 02), persisten dos grandes desafíos operativos en una organización empresarial de retail:

1. **Conexión en Vivo a Sistemas mediante Model Context Protocol (MCP)**:

Ni los ingenieros ni los gerentes comerciales de categoría desean copiar y pegar consultas SQL o archivos CSV en ventanas de chat. En su lugar, Antigravity debe consultar directamente la base de datos en vivo **`data/retail_pulse.db`** mediante herramientas estructuradas de **Model Context Protocol (MCP)** para inspeccionar el True ATP por nodo, auditar contratos OTIF de proveedores en incumplimiento, simular márgenes promocionales antes de su lanzamiento y listar pedidos VIP en riesgo de quiebre de stock.

2. **Guardrails Deterministas y Gobernanza Empresarial**:

Las instrucciones en **`.agents/rules/`** guían el razonamiento del LLM, pero el cumplimiento de misión crítica —como bloquear que alguien desactive **`enforce_floor=False`** en **`src/services/promo_pricing.py`** o codifique **`minimum_margin_pct`** por debajo de **`0.15`** (**15%**)— requiere **ejecución determinista** fuera de la salida probabilística del LLM, junto con **gobernanza en Google Cloud / Gemini Enterprise** y **controles de Sandbox y Permisos del cliente Antigravity**.

2. Objetivos de Aprendizaje

En este laboratorio lograrás:

1. Completar el **Servidor FastMCP de Retail Pulse** (**`mcp_server/retail_mcp.py`**) implementando dos herramientas MCP de alto impacto:

- **`simulate_promo_margin_guard`**: Simula la aplicación acumulada de una campaña promocional sobre un SKU o categoría e informa si algún SKU incumple su piso mínimo de margen.
- **`list_at_risk_vip_orders`**: Consulta **`omnichannel_orders`** para pedidos **`AT_RISK_STOCKOUT`** filtrados por nivel de lealtad (**`DIAMOND`**, **`GOLD`**) y los ordena por Customer Lifetime Value (**`customer_ltv_usd`**).

2. Registrar el servidor MCP local en **`.agents/mcp_config.json`** para que Antigravity descubra e invoque automáticamente las 4 herramientas MCP de retail.

3. Configurar un hook determinista de ciclo de vida **`PreToolUse`** en **`.agents/hooks.json`** respaldado por **`scripts/margin_guard_hook.py`** que intercepta cualquier llamada a **`write_file`** / **`edit_file`** que viole el piso de margen del 15% y escribe un registro de auditoría inmutable en **`data/logs/hook_compliance_audit.jsonl`**.
4. Recorrer la arquitectura de **Gobernanza de Google Cloud & Gemini Enterprise** y los **Controles de Seguridad del Cliente y Workspace de Antigravity** para despliegues empresariales de retail.

3. Instrucciones Paso a Paso

Paso 1: Inicializar el Estado Inicial del Laboratorio 03

Ejecuta el instalador del Laboratorio 03 para asegurar que las soluciones de los Laboratorios 01 y 02 estén activas y cargar los archivos iniciales del Laboratorio 03:

```
./labs/lab-03-mcp-and-hooks/install.sh
```

Verifica que la suite de pruebas del Laboratorio 03 falle inicialmente en las herramientas MCP aún no implementadas y los archivos faltantes **`.agents/mcp_config.json`** / **`.agents/hooks.json`**:

```
./labs/lab-03-mcp-and-hooks/verify.sh
```

Paso 2: Inspeccionar el Servidor MCP Inicial (`mcp_server/retail_mcp.py`)

Abre **`mcp_server/retail_mcp.py`**. Observa que dos herramientas MCP ya están implementadas:

- **`query_true_atp_inventory(sku: str = "", category: str = "") -> str`**
- **`audit_vendor_otif_contracts(only_breaching: bool = True) -> str`**

Sin embargo, dos herramientas son actualmente stubs que devuelven **`{"status": "NOT_IMPLEMENTED"}`**:

- **`simulate_promo_margin_guard(promo_code: str, category: str = "", loyalty_tier: str = "SILVER") -> str`**
- **`list_at_risk_vip_orders(loyalty_tier: str = "DIAMOND", limit: int = 20) -> str`**

Paso 3: Implementar las Herramientas MCP Restantes y Registrar `.agents/mcp_config.json`

Abre **`labs/lab-03-mcp-and-hooks/PROMPTS.es.md`** y utiliza los **Prompts de Implementación** (o **Edición en Línea `Cmd+I` / `Ctrl+I`**) para:

1. Implementar **`simulate_promo_margin_guard`** en **`mcp_server/retail_mcp.py`** para que evalúe todos los SKUs de la categoría solicitada contra el **`promo_code`** y **`loyalty_tier`** usando **`evaluate_sku_promo()`**, devolviendo **`total_skus_evaluated`**, **`floor_violations_count`**, **`requires_vp_merchandising_override`** y el detalle por SKU (**`sku`**, **`product_name`**, **`unit_cost_usd`**, **`msrp_usd`**, **`effective_price_usd`**, **`margin_pct`**, **`minimum_margin_pct`**, **`floor_clamped`**).
2. Implementar **`list_at_risk_vip_orders`** en **`mcp_server/retail_mcp.py`** para que consulte **`omnichannel_orders`** donde **`status = 'AT_RISK_STOCKOUT'`** y **`customer_loyalty_tier = loyalty_tier`** (o todos los niveles si **`loyalty_tier == "ALL"`**), ordenados por **`customer_ltv_usd DESC`**, devolviendo **`count`**, **`total_ltv_exposed_usd`** y **`orders`**.
3. Crear **`.agents/mcp_config.json`** registrando el servidor **`retail-pulse-mcp`** mediante **`.venv/bin/python -m mcp_server.retail_mcp`**.

Paso 4: Configurar el Hook Determinista `PreToolUse` de Protección de Margen (`.agents/hooks.json`)

Inspecciona **`scripts/margin_guard_hook.py`**. Este script lee el payload de la llamada a herramienta desde **`stdin`** antes de que ocurra cualquier modificación de archivo:

- Si una llamada a herramienta intenta establecer **`enforce_floor=False`** o **`enforce_floor = False`** dentro de **`src/services/promo_pricing.py`**, o establece **`"minimum_margin_pct"`** por debajo de **`0.15`**, el hook finaliza con código de salida **`2`** (**`BLOCKED`**) e imprime un mensaje claro de incumplimiento en **`stderr`**.
- Cada evaluación del hook (**`ALLOW`** o **`BLOCK`**) se agrega como un evento JSON estructurado en **`data/logs/hook_compliance_audit.jsonl`**.

Crea **`.agents/hooks.json`** para conectar **`scripts/margin_guard_hook.py`** al evento de ciclo de vida **`PreToolUse`** para **`write_file`**, **`edit_file`** y **`replace_file_content`**.

Prueba el hook directamente desde la terminal para verificar tanto el bloqueo como el registro en el log de auditoría:

```
echo '{"tool_name":"edit_file","tool_input":{"Path":"src/services/promo_pricing.py","Rep \
  lacement":"enforce_floor=False"}}' | .venv/bin/python scripts/margin_guard_hook.py
cat data/logs/hook_compliance_audit.jsonl
```

4. Inmersión Profunda en Seguridad, Privacidad y Gobernanza Empresarial

Al desplegar Antigravity en una empresa de retail (merchandising, cadena de suministro, operaciones en tienda e ingeniería de plataforma), los arquitectos de seguridad gobiernan la plataforma en **dos capas complementarias**:

```
+-------------------------------------------------------------------------------+
| CAPA 1: Gobernanza en Google Cloud & Gemini Enterprise (Plano de Admin Cloud)  |
|  - IAM e Identidad: roles/cloudaicompanion.user vía Cloud Identity / SSO Corp  |
|  - Cloud Audit Logs: cloudaicompanion.googleapis.com en Google Cloud Logging   |
|  - Privacidad: Cero Retención de Datos y Sin Entrenamiento con prompts/código  |
|  - Perímetro de Red: VPC Service Controls (VPC-SC) y Endpoints Regionales      |
|  - Protección IP: Indemnización de Google Cloud y Filtros de Citación          |
+-------------------------------------------------------------------------------+
                                    |
                                    v
+-------------------------------------------------------------------------------+
| CAPA 2: Controles de Seguridad del Cliente y Workspace Antigravity (Plano Local)  |
|  - Política de Ejecución: strict | request-review | proceed-in-sandbox          |
|  - Sandbox de Terminal y Aislamiento de Red: Sandboxing a nivel de SO          |
|  - Acceso a Archivos Fuera del Workspace: Restricción estricta al repositorio  |
|  - Lista Blanca/Negra de Comandos y Lista Blanca del Navegador                 |
|  - Hooks Deterministas de Ciclo de Vida: .agents/hooks.json -> auditoría JSONL |
+-------------------------------------------------------------------------------+
```

Capa 1: Gobernanza en Google Cloud & Gemini Enterprise

Área de Control	Capacidad en Google Cloud / Gemini Enterprise	Caso de Uso en Retail Empresarial
IAM y Gestión de Licencias	Asignar **`roles/cloudaicompanion.user`** mediante Google Cloud IAM integrado con grupos SSO de **Cloud Identity / Okta / Azure AD (Entra ID)**.	Otorgar acceso diferenciado a **`retail-platform-eng@`** y **`retail-commercial-ops@`** con alta/baja centralizada.
Cloud Audit Logging	Habilitar logs de auditoría **Admin Activity** y **Data Access** para **`cloudaicompanion.googleapis.com`** en **Google Cloud Logging**, y enrutarlos mediante Log Sinks a BigQuery o Chronicle SIEM.	Mantener una pista de auditoría empresarial inmutable de quién invocó Gemini Enterprise, desde qué IP/dispositivo y cuándo.
Privacidad de Datos y Cero Entrenamiento	Los prompts de clientes, contexto del workspace, esquemas SQL y código generado **nunca se utilizan para entrenar modelos fundacionales compartidos** y cumplen garantías de cero retención fuera de tu tenant.	Analizar de forma segura contratos confidenciales de proveedores (**`data/exports/vendor_contracts.csv`**), costos unitarios mayoristas y LTV de clientes VIP.
VPC Service Controls (VPC-SC)	Ubicar **`cloudaicompanion.googleapis.com`** dentro de un **perímetro de VPC Service Controls** junto con BigQuery, Cloud SQL y Secret Manager, con fijación de endpoints regionales.	Prevenir la exfiltración de datos desde estaciones de trabajo corporativas o redes externas no autorizadas.

Capa 2: Controles de Seguridad del Cliente y Workspace de Antigravity

Configuración / Hook del Cliente	Opciones de Configuración	Configuración Recomendada en Producción Retail
Política de Ejecución de Herramientas	**`strict`**, **`request-review`**, **`proceed-in-sandbox`**, **`always-proceed`**	Usar **`proceed-in-sandbox`** o **`request-review`** para que los comandos mutantes requieran sandboxing del SO o aprobación humana.
Sandbox de Terminal y Política de Red	Sandbox a nivel de SO (**`sandbox-exec`** en macOS / namespaces en Linux) + Interruptor de Acceso a Internet	Habilitar el Sandbox de Terminal y restringir llamadas de red externas para alcanzar solo endpoints internos aprobados.
Acceso a Archivos Fuera del Workspace	Bloquear o requerir confirmación para lecturas/escrituras fuera de la raíz del workspace	**Bloquear** el acceso fuera del workspace para impedir lecturas de **`~/.ssh`**, **`~/.aws`** u otros directorios del host.
Lista Blanca / Negra de Comandos	Lista blanca de prefijos (**`pytest`**, **`.venv/bin/python`**) y lista negra (**`rm -rf`**, **`curl`**, **`nc`**)	Permitir **`.venv/bin/python -m pytest`** para TDD autónomo bloqueando comandos destructivos o de exfiltración.
Lista Blanca del Navegador	Lista blanca de dominios para el subagente **`/browser`**	Restringir **`/browser`** estrictamente a **`http://127.0.0.1:8000`** y dashboards internos aprobados.

Configuración / Hook del Cliente	Opciones de Configuración	Configuración Recomendada en Producción Retail
Hooks Deterministas de Ciclo de Vida	`.agents/hooks.json` (`PreToolUse`, `PostToolUse`)	Hacer cumplir invariantes comerciales y de seguridad (como `scripts/margin_guard_hook.py`) y registrar cada verificación en `data/logs/hook_compliance_audit.jsonl`.

5. Lista de Verificación

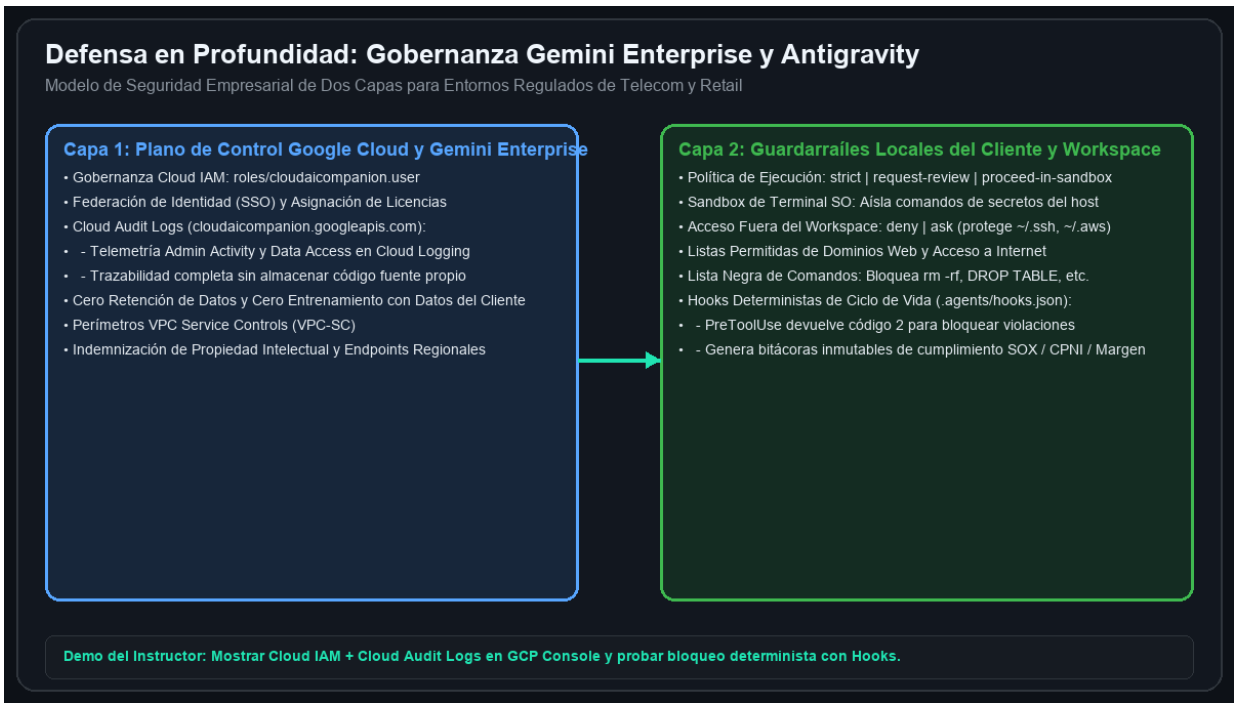
Ejecuta el script de verificación del Laboratorio 03 para confirmar que tus herramientas MCP, `.agents/mcp_config.json` y `.agents/hooks.json` pasan todas las pruebas automatizadas:

```
./labs/lab-03-mcp-and-hooks/verify.sh
```

Resultado esperado:

- `tests/test_lab03_mcp_hooks.py` pasa **5/5 pruebas**:
- `test_mcp_query_true_atp_inventory`
- `test_mcp_audit_vendor_otif_contracts`
- `test_mcp_simulate_promo_margin_guard`
- `test_mcp_list_at_risk_vip_orders`
- `test_mcp_config_and_margin_guard_hook`

¿Necesitas saltar directamente al estado verificado? Ejecuta `./labs/lab-03-mcp-and-hooks/apply_solution.sh`.



LAB 03 PROMPTS OPTIMIZADOS PARA COPIAR Y PEGAR (ES)
Delivered by Tryolabs • Powered by Google Antigravity & Gemini Enterprise

Playbook de Prompts del Laboratorio 03: Servidor MCP de Retail y Hooks Deterministas de Margen

Copia y pega estos prompts en **Google Antigravity** a medida que avanzas en el Laboratorio 03. Cada sección incluye prompts diseñados tanto para el **Track Técnico** como para el **Track Comercial / Merchandising**.

Fase 1: Descubrimiento e Inspección de Arquitectura

Prompt 1A — Track Técnico: Inspeccionar el Servidor MCP Inicial y el Script del Hook

```
Lee @mcp_server/retail_mcp.py, @scripts/margin_guard_hook.py y @tests/test_lab03_mcp_hoo \
  ks.py.
Resume:
1. Cómo las dos herramientas MCP existentes (`query_true_atp_inventory` y `audit_vendor_ \
  otif_contracts`) se conectan a `data/retail_pulse.db` y a nuestra capa de servicios.
2. Qué campos JSON exactos deben devolver `simulate_promo_margin_guard` y `list_at_risk_ \
  vip_orders` para pasar `tests/test_lab03_mcp_hooks.py`.
3. Cómo `scripts/margin_guard_hook.py` inspecciona los payloads `PreToolUse` en stdin y  \
  escribe eventos de cumplimiento en `data/logs/hook_compliance_audit.jsonl`.
```

Prompt 1B — Track Comercial / Operaciones: Comprender MCP en Vivo vs. Hooks Deterministas

```
Lee @mcp_server/retail_mcp.py y @scripts/margin_guard_hook.py. Explica en términos comer \
  ciales de retail:
1. Cómo las 4 herramientas MCP permiten a un gerente de Merchandising o Supply Chain con \
  sultar en vivo el stock True ATP, los contratos OTIF en incumplimiento, el impacto pro \
  mocional en márgenes y los pedidos VIP en riesgo sin escribir SQL.
2. Por qué combinamos nuestras directrices en `.agents/rules/retail-standards.md` con un \
   hook determinista `PreToolUse` (`scripts/margin_guard_hook.py`) que bloquea cualquier \
   intento de desactivar el piso de margen del 15%.
```

Fase 2: Planificación (`/plan`)

Prompt 2 — Crear un Plan de Implementación para Herramientas MCP y Hooks de Ciclo de Vida

```
/plan Diseña la implementación del Laboratorio 03 para que pasen las 5 pruebas en @tests \
  /test_lab03_mcp_hooks.py:
1. Implementar `simulate_promo_margin_guard(promo_code: str, category: str = "", loyalty \
  _tier: str = "SILVER") -> str` y `list_at_risk_vip_orders(loyalty_tier: str = "DIAMOND \
  ", limit: int = 20) -> str` en @mcp_server/retail_mcp.py.
2. Crear `.agents/mcp_config.json` registrando `retail-pulse-mcp` usando `.venv/bin/pyth \
  on -m mcp_server.retail_mcp`.
3. Crear `.agents/hooks.json` registrando `.venv/bin/python scripts/margin_guard_hook.py \
```

```
` como un hook `PreToolUse` para `write_file|edit_file|replace_file_content`.
```

Fase 3: Edición en Línea (`Cmd+I` / `Ctrl+I`)

Abre **`mcp_server/retail_mcp.py`**, selecciona las funciones stub **`simulate_promo_margin_guard`** y **`list_at_risk_vip_orders`**, presiona **`Cmd+I`** (macOS) o **`Ctrl+I`** (Windows/Linux) e ingresa:

```
Implementa `simulate_promo_margin_guard` usando `evaluate_sku_promo` de `src.services.pr \
  omo_pricing` (con `enforce_floor=True`) para que devuelva JSON con `promo_code`, `loya \
  lty_tier`, `category_filter`, `total_skus_evaluated`, `floor_violations_count`, `requi \
  res_vp_merchandising_override` y `skus`. Luego implementa `list_at_risk_vip_orders` co \
  nsultando `omnichannel_orders` donde `status = 'AT_RISK_STOCKOUT'` ordenado por `custo \
  mer_ltv_usd DESC`, devolviendo JSON con `loyalty_tier_filter`, `count`, `total_ltv_exp \
  osed_usd` y `orders`.
```

Fase 4: Prompts de Implementación Completa

Prompt 4A — Implementar `mcp_server/retail_mcp.py`, `.agents/mcp_config.json` y `.agents/hooks.json`

```
Completa el Laboratorio 03 realizando los siguientes cambios:
1. En @mcp_server/retail_mcp.py:
   - Implementa `simulate_promo_margin_guard(promo_code: str, category: str = "", loyalt \
  y_tier: str = "SILVER") -> str` consultando `wms_inventory` para los SKUs distintos (f \
  iltrados por `category` si se proporciona), llamando a `evaluate_sku_promo(conn, sku,  \
  promo_code=promo_code, loyalty_tier=loyalty_tier, enforce_floor=True)` y devolviendo J \
  SON con `promo_code`, `loyalty_tier`, `category_filter`, `total_skus_evaluated`, `floo \
  r_violations_count`, `requires_vp_merchandising_override` y `skus`.
   - Implementa `list_at_risk_vip_orders(loyalty_tier: str = "DIAMOND", limit: int = 20) \
   -> str` consultando `omnichannel_orders` donde `status = 'AT_RISK_STOCKOUT'` (y `cust \
  omer_loyalty_tier = ?` a menos que `loyalty_tier.upper() == "ALL"`), ordenado por `cus \
  tomer_ltv_usd DESC LIMIT ?`, y devolviendo JSON con `loyalty_tier_filter`, `count`, `t \
  otal_ltv_exposed_usd` y `orders`.
2. Crea `.agents/mcp_config.json` con una entrada `mcpServers` para `retail-pulse-mcp` e \
  jecutando `.venv/bin/python -m mcp_server.retail_mcp`.
3. Crea `.agents/hooks.json` con un hook `PreToolUse` que coincida con `write_file|edit_ \
  file|replace_file_content` y ejecute `.venv/bin/python scripts/margin_guard_hook.py`.
```

Fase 5: Verificación y Consultas MCP en Vivo

Prompt 5A — Ejecutar la Verificación Automatizada (`verify.sh`)

```
Ejecuta `./labs/lab-03-mcp-and-hooks/verify.sh`, confirma que las 5 pruebas en `tests/te \
  st_lab03_mcp_hooks.py` pasen y muestra la última entrada en `data/logs/hook_compliance \
  _audit.jsonl`.
```

Prompt 5B — Track Comercial: Consultar las Herramientas MCP de Retail para Insights Ejecutivos

```
Usando las funciones en @mcp_server/retail_mcp.py, responde estas tres preguntas comerci \
  ales:
1. Si acumulamos la promoción `VIP-EXTRA-15` para miembros `DIAMOND` en la categoría `El \
  ectronics`, ¿cuántos SKUs activan el límite del piso de margen del 15%?
2. ¿Cuántos pedidos VIP `DIAMOND` están actualmente en estado `AT_RISK_STOCKOUT` y cuál  \
  es el Customer LTV total expuesto?
```

```
3. ¿Qué proveedores en incumplimiento tienen un puntaje OTIF inferior al 85% y cuál es s \
  u penalización combinada por chargeback en USD?
```

Fase 6: Desafío Adicional — Probar el Hook de Margen ante Prompts Adversarios

Prompt 6 — Intentar Evadir el Piso de Margen del 15% (Observar el Bloqueo del Hook)

```
Simula una solicitud no autorizada: prueba ejecutar `scripts/margin_guard_hook.py` contr \
  a un payload que intente editar `src/services/promo_pricing.py` con `enforce_floor=Fal \
  se`, y otro payload que intente establecer `"minimum_margin_pct": 0.05`. Muestra los c \
  ódigos de salida del hook, los mensajes en stderr y las entradas registradas en `data/ \
  logs/hook_compliance_audit.jsonl`.
```